Lab Guide 3

Introduction to compute accelerators: Vector Processing and Roofline Model

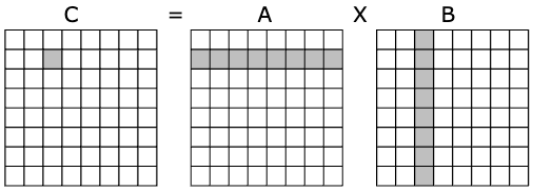
At the end of this lab session students should be able to:

- Quantify the potential performance benefits of vector processing and main limitations
- Understand the trade-off between performance and floating point precision
- Use of the roofline model to identify hardware limitations (memory bottleneck)

Introduction

The peak performance of modern machines can only be attained when vector processing is explored. However, vector processing can only be applied to certain kinds of applications (e.g., data parallelism, where the same set of operations can applied to multiple elements) and requires data structures with spatial locality. Moreover, applications using vector processing consume more data per second, becoming more constrained by memory bandwidth. The roofline model is a visual model that shows the peak performance and how memory bandwidth constrains the system's performance for a particular algorithm.

```
for(int i=0; i<N; i++) {
  for(int j=0; j<N; j++) {
    for(int k=0; k<N; k++) {
      C[i*N+j] += A[i*N+k] * B[k*N+j];
    }
  }
}
```



Exercise 1 – Compiler- based vectorisation

a) Identification of limitations**:** The compiler cannot vectorise the "base" DOT matrix multiplication code. Identify two reasons for this (suggestion: look at the matrix data access patterns in the DOT code, namely, reads from A; reads from B and writes to C).

b) Estimation of vectorization impact: The order of the three loops can be changed without affecting the correctness of the results. Identify one (or more) order of the loops that directly enable vectorisation. What is the potential gain, in number of instruction, for vectors of size 2 and for size 4?

c) Enabling vectorisation: Implement the version that you selected and compile the code adding the options `-ftree-vectorize -msse4` (the second flag generates SSE4 vector instructions which supports vector operations of 128bits). How many elements are processed at once? What is the potential gain by using vector processing?

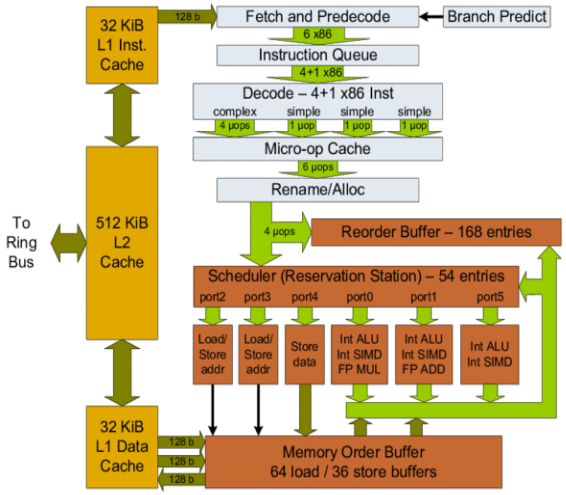
d) Measure and discuss the results: Complete the table and comment the results (is the performance gain what was expected? Why?).

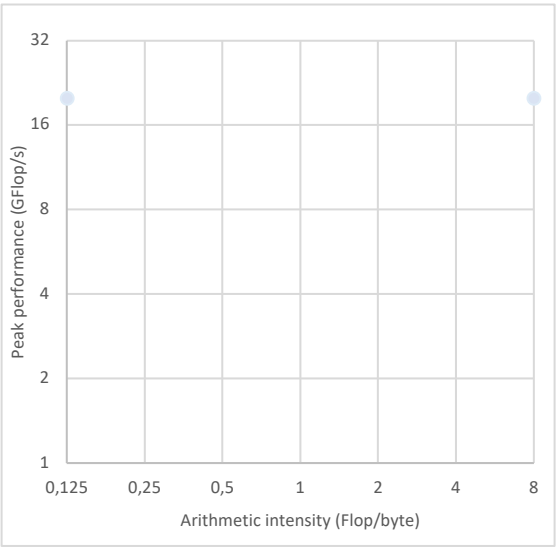
N	Version	Time	CPI	#I (inst_retired.any)
512	base_v()			
	vect() double			
	vect() float			

e) Change the matrix data type from `double` to `float`. What is the expected gain of this change? Compete the table and explain the results.

f) (*) Vectorisation fine-tuning: In order to further improve vectorisation you can compile with `-mavx` to use vector operations on 256bit vectors. However to take advantage of this vector size the data should be aligned at 32-byte memory addresses (Why?). On way to enforce this alignment in `gcc` is to declare the vector statically with the aligned attribute (e.g., `double A[size*size] __attribute__ ((aligned (32)));` etc.).

Exercise 2 - Roofline Model





a) Peak Performance: Look at the figure above. How many operations can be performed at each cycle in this architecture? Identify, from the figure, constraints among floating point operations that can run simultaneously. What is the floating point peak performance (with double data type), in GFlop per second, if the machine runs at 2.5 GHz and vector size is 256bits. How much time does it take to perform an 1024x1024 matrix multiplication at this GFlop/s rate?

b) Complete the graph, at the right-side of the figure, with this GFlop/s in the Y-axis by plotting a horizontal line representing this limit.

c) Arithmetic Intensity is the ratio of total floating-point operations to total data movement (bytes). Assuming a machine with a memory bandwidth 20 GB/s, what is the lowest arithmetic intensity supported by this machine without being constrained by the memory bandwidth (note: the required memory bandwidth for a given arithmetic intensity can be obtained by dividing the peak performance by the arithmetic intensity). Write this point in the X-axis.

d) Memory Bandwidth Limitation: What happens if the application requests more bytes per floating point operation than the available memory bandwidth? Plot a line in the graph showing this limit.

e) (*) Roofline model applied to the matrix multiplication: Look at the code of the `ikj` variant of the matrix multiplication. How many floating point operations are performed per byte read from memory ? Draw this point in the figure in order to estimate the attainable performance of this implementation.

```
/* Cij =0 */
for i=0 to N-1
     for k=0 to N-1
          for j=0 to N-1
               C[i][j] += A[i][k] * B[k][j]
```

f) (*) Draw the roofline model for the matrix multiplication execution on your personal computer (vectorised variant). You can use the STREAM benchmark the get the memory bandwidth (or get it from the machine's specification). You can also apply the *parallel* loop unrolling technique (see lab 2, exercise 2a)) to avoid the memory bottleneck and measure the performance gain on a large matrix (e.g., of 1024x1024 or larger).